Context-Aware Deadline Scheduling Under Memory Contention for Embedded Real-Time Systems

Eren Karadağ* and Umut Kalman†
*Computer Engineering Department, Hacettepe University, Ankara, Türkiye
†Graduate School of Science and Engineering, Hacettepe University, Ankara, Türkiye
E-mails: *erenkaradag@hacettepe.edu.tr, †umutkalman@hacettepe.edu.tr

***Abstract*—Modern embedded systems are increasingly required to handle complex, mixed-criticality workloads where computation-bound control loops are run together with data-intensive processing tasks. In these constrained environments, memory contention has come out as a critical factor threatening real-time determinism. Although standard scheduling algorithms, such as Rate Monotonic (RM) and Earliest Deadline First (EDF), optimize primarily for CPU utilization and deadline compliance, they often fail to account for the overhead introduced by memory interference. This project presents a context-aware scheduling strategy designed to reduce these penalties in real-time embedded systems.**

We propose a lightweight, user-space scheduler wrapper implemented on FreeRTOS that extends the classic EDF algorithm with a "Slack-Aware" heuristic. This mechanism dynamically evaluates the memory intensity of pending tasks and introduces a penalty cost model with metrics to predict contention. With task slack times in mind, the scheduler reorders execution sequences to minimize consecutive memory-heavy operations without violating hard deadlines. To validate the effectiveness of this online heuristic, first we develop an offline Ant Colony Optimization (ACO) model that determines the theoretical optimal schedule for a given task set. Then, we deploy our model in an application running on a microcontroller with a single-core processor to examine the custom scheduler we developed.

Experimental results compare the proposed slack-aware EDF wrapper scheduler against standard pure EDF scheduling as baseline, using the ACO solution as a performance baseline. The study demonstrates that the proposed context-aware scheduler significantly reduces simulated memory penalties and context-switch overheads compared to traditional EDF, while maintaining the determinism required for embedded real-time applications.

***Index Terms*—Real-time embedded systems, memory contention, Earliest Deadline First (EDF) scheduling, slack-aware scheduling, Ant Colony Optimization (ACO).**

I. Introduction

Real-time systems are foundational to safety-critical infrastructure, including automotive control units, defense systems, robotics, avionics, and medical devices. In these critical environments, timing predictability is naturally tied to overall system safety and reliability; a missed deadline in a critical component, such as a braking controller or flight control unit, can lead to unstable operation, severe physical damage, or even life-threatening consequences. As modern embedded systems evolve, they are increasingly required to handle complex, mixed-criticality workloads that run control loops alongside data-intensive processing tasks. With platforms becoming more memory-intensive, shared memory contention has emerged as a critical threat to real-time determinism.

While classical scheduling theory provides strong guarantees under ideal assumptions, real-world embedded environments are significantly more complex. Standard scheduling algorithms, such as Rate Monotonic (RM) and Earliest Deadline First (EDF), are highly effective at optimizing CPU utilization and deadline compliance. However, they typically fail to account for the hidden execution overheads introduced by memory interference. This oversight creates a distinct gap between theoretical schedulability and observed runtime behavior. In practice, engineers frequently encounter systems that are proven schedulable on paper but still suffer from jitter, unpredictable latency, and sporadic deadline misses when deployed due to this hidden resource interference.

To address these challenges, this paper introduces a context-aware scheduling strategy designed to reduce memory contention penalties in real-time embedded systems without requiring major system redesigns or expensive hardware upgrades. We propose a lightweight, user-space scheduler wrapper implemented on FreeRTOS that extends the classic EDF algorithm with a dynamic slack-aware heuristic. This mechanism evaluates the memory intensity of pending tasks at runtime and utilizes a penalty cost model to predict contention. By leveraging available task slack times, the scheduler intelligently reorders execution sequences to separate consecutive memory-heavy operations, thereby minimizing interference while strictly maintaining hard deadlines.

To thoroughly validate the effectiveness of this approach, the project pairs the online heuristic with an offline Ant Colony Optimization (ACO) model, which generates a theoretical optimal schedule to serve as a performance baseline. We will deploy the model on a single-core STM32L4-series microcontroller to examine the custom scheduler's real-world performance, comparing it against standard static priority and pure EDF scheduling methodologies. Ultimately, the goal is to move one step closer to real-time systems that behave in the field as predictably as they do in theory, improving safety, trust, and efficiency across embedded computing domains.

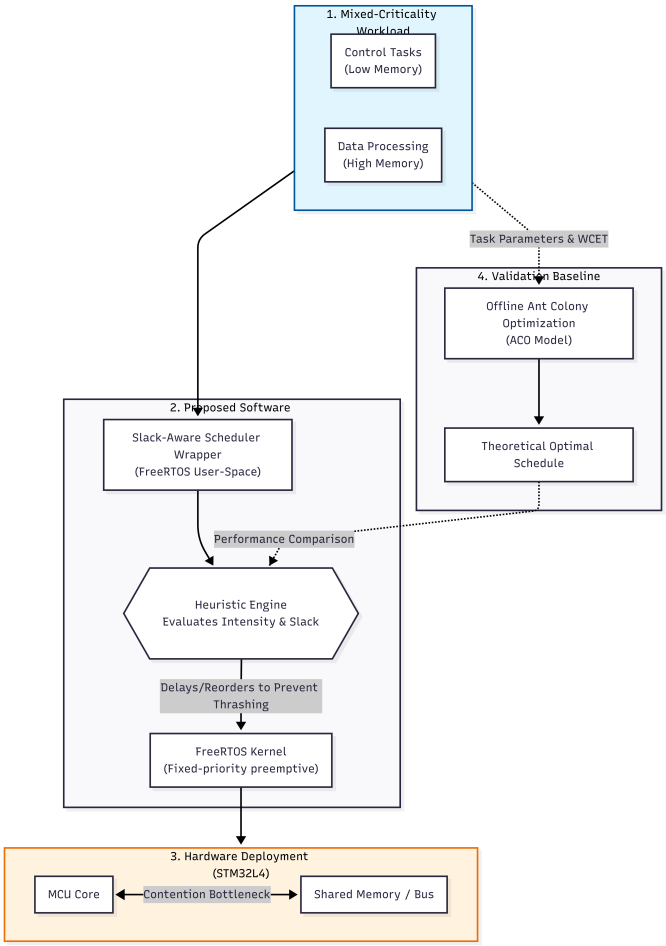

Fig. 1. Proposed custom scheduler wrapper implementation flow

II. Related Work

EDF remains the classical baseline for uniprocessor real-time scheduling because, under ideal assumptions, it is optimal in processor utilization [1]. In practice, however, those assumptions hide preemption and memory-system overheads, so timing behavior can deviate from theoretical guarantees.

Prior research addressed memory interference through system-level control and scheduling structure. OS-level methods such as MemGuard and bank-aware allocation improve isolation by regulating bandwidth and memory placement [2], [3]. PREM-style and memory-centric approaches make memory phases scheduler-visible to improve predictability [4]–[6], but often require explicit phase boundaries and stronger runtime support.

Another line of work extends EDF analysis by modeling cache-related preemption delay (CRPD), showing that cache effects can significantly affect schedulability [7]. Slack reclaiming studies also show that online slack can improve EDF behavior [8], though they mainly target soft real-time optimization rather than memory-aware hard real-time ordering.

Finally, online/offline studies with preemption penalties show that no universally optimal online policy exists in the general case, while offline optimization can provide stronger references [9]. Motivated by this, our work combines a lightweight online EDF wrapper with an offline optimization baseline, using memory intensity and slack jointly without heavy kernel changes or specialized hardware.

III. Problem Definition

A. System / Application Description

The target application is a mixed-critical embedded real-time system, representative of domains such as automotive electronic control units (ECUs) and mission-critical defense platforms. In this setting, the system must execute two workload classes concurrently:

- **Control tasks (low memory intensity):** high-frequency, hard real-time activities for actuation, sensing, and motor-control loops. These tasks are primarily computation-bound and require strict deadline compliance.
- **Data-intensive tasks (high memory intensity):** lower-frequency but high-bandwidth operations, such as signal processing, image filtering, and cryptographic payload handling. These tasks require frequent access to shared memory structures.

When these workload classes are interleaved, memory-intensive executions can increase bus and cache interference, which appears as jitter and unpredictable delay in critical control jobs.

In this study the representative workload contains three periodic tasks:

- τ_1 (motor control, low memory intensity): a high-frequency hard real-time control task (e.g., PID loop), primarily computation-bound.
- τ_2 (sensor data acquisition, low memory intensity): periodic ADC sampling and filtering used by the control pipeline.
- τ_3 (cryptographic encryption, high memory intensity): lower-frequency telemetry encryption with heavy memory access.

When these tasks are interleaved, memory-intensive executions can increase cache and bus interference, introducing jitter for time-critical control jobs.

1) System Context and Processor Model:
The scheduler wrapper runs on FreeRTOS over an ARM Cortex-M4 STM32L4-series microcontroller. Because execution is single-core, concurrency is realized by preemption and context switching, so contention appears as sequential interference between jobs with different memory-access profiles.

2) Assumptions and Task Model:
The task model uses the tuple $(\mathbf{C_i}, \mathbf{T_i}, \mathbf{D_i}, \mathbf{M_i})$ for each periodic task τ_i, where C_i is isolated WCET, T_i is period, D_i is deadline, and $M_i \in [0, 1]$ is normalized memory intensity. Tasks are independent and non-blocking. Baseline EDF schedulability is assumed under isolation:

$$\sum_{i=1}^{3} \frac{C_i}{T_i} \leq 1.$$

TABLE I
COMPARISON OF THE MOST RELEVANT STUDIES AND THE POSITION OF THIS WORK.

Work	Year	Multi-core	EDF	Mem/Cache Aware	Slack	Online	Offline Oracle	Heavy Support
Liu & Layland [1]	1973		✓			✓		
MemGuard [2]	2013	✓		✓		✓		✓
PALLOC [3]	2014	✓		✓		✓		✓
PREM [4]	2011	✓		✓				✓
MCS-RTOS [5]	2019	✓		✓		✓		✓
FP-MCS [6]	2020	✓		✓		✓		✓
EDF+CRPD [7]	2013		✓	✓				
Slack reclaiming [8]	2005		✓		✓	✓		✓
Online/Offline CRPD [9]	2018		✓	✓		✓	✓	
This Work	2026		✓	✓	✓	✓	✓	

B. Objectives

1) Implement standard EDF scheduling as baseline
2) Minimize contention penalties by ordering executions to reduce cumulative memory-interference overhead.
3) Maintain real-time guarantees by targeting zero deadline misses for theoretically schedulable task sets.
4) Establish a deterministic baseline using an offline Ant Colony Optimization (ACO) schedule as ground truth for comparison.

C. Constraints

1) **Timing:** Missing any absolute deadline d_j is treated as a critical failure.
2) **Hardware:** Single-core shared-memory execution implies sequential preemption penalties; total CPU utilization must remain at or below 100%.
3) **Runtime overhead:** The online FreeRTOS wrapper must remain lightweight so decision overhead does not exceed contention-related savings.

D. Mathematical Formulation (Optimization Track)

Let $\mathcal{J} = \{j_1, j_2, \ldots, j_N\}$ denote all released jobs in one hyperperiod. Define binary decision variable

$$x_{a,b} = \begin{cases} 1, & \text{if } j_b \text{ immediately follows } j_a, \\ 0, & \text{otherwise.} \end{cases}$$

The pairwise memory penalty is

$$P(j_a, j_b) = \alpha\, M_a\, M_b.$$

The objective is

$$\min \sum_{a=1}^{N} \sum_{b=1}^{N} x_{a,b}\, P(j_a, j_b).$$

a) Sequence Integrity:

$$\sum_{b=1,\, b\neq a}^{N} x_{a,b} = 1 \quad \forall a \in \mathcal{J},$$

$$\sum_{a=1,\, a\neq b}^{N} x_{a,b} = 1 \quad \forall b \in \mathcal{J}.$$

b) Deadline Compliance:

$$f_b = s_b + C_b + \sum_{a=1}^{N} \left(x_{a,b}\, P(j_a, j_b) \right),$$

$$f_b \leq d_b \quad \forall b \in \mathcal{J}.$$

c) Online Slack Proxy:

$$Slack_j(t) = d_j - t - C_{rem,j}(t).$$

High-memory tasks are deferred only when $Slack_j(t) > 0$.

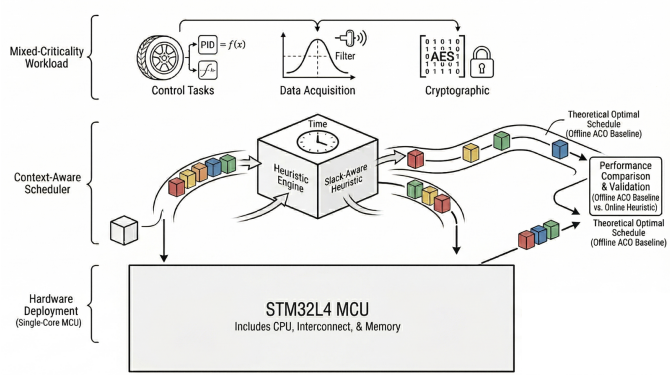

Fig. 2. Tasks running concurrently on MCU

IV. System Architecture and Design

A. Hardware/Software Architecture

The implementation combines a resource-constrained embedded platform with a lightweight scheduling layer:

- **Hardware platform:** STM32L4-series microcontroller (ARM Cortex-M4), unified memory bus, and on-chip SRAM.
- **Operating system:** FreeRTOS with baseline preemptive tick scheduling.
- **Scheduler wrapper:** custom user-space C module that inspects ready tasks and adjusts execution order based on slack-aware contention heuristics.

B. Dataflow and System Model

System execution starts with periodic hardware-timer interrupts that release jobs into the FreeRTOS ready queue. Instead of always dispatching the earliest-deadline task immediately, the wrapper evaluates the memory intensity of the running and pending jobs. When predicted pairwise contention is high, the wrapper checks dynamic slack and defers the memory-intensive job only if timing slack remains positive.

C. Timing and Execution Model

The system follows a periodic, event-driven model implemented with ‘vTaskDelayUntil()‘. The base dispatching rule is EDF with dynamic deadlines. The wrapper applies bounded, slack-aware reordering to reduce contention without violating real-time constraints.

D. Model of Computation

The design is modeled as a discrete-event system with synchronous periodic releases. Scheduling events are triggered by job releases, job completions, and contention-aware wrapper checks.

E. Assumptions and Design Limitations

The current design assumes accurate offline profiling of WCET (C_i) using ARM DWT counters. It also treats memory intensity (M_i) as a static task-level characteristic; fine-grained intra-task phase variation is not explicitly modeled.

F. Resource Model

The proposed scheduler is designed under strict embedded resource limits of a single-core STM32L4-series platform. These limits directly motivate the contention-aware design.

1) Memory Usage (RAM and Buffers): The platform provides limited SRAM (typically 128–192 KB). A fixed portion is reserved by FreeRTOS, including task control blocks (TCBs) and task stacks. Control-oriented tasks (τ_1, τ_2) use small local buffers, whereas the cryptographic task (τ_3) uses larger contiguous buffers (approximately 4–10 KB).

Since τ_3 drives high memory traffic, the scheduler wrapper must remain lightweight. Runtime dynamic allocation is avoided, and task metadata lookup is designed to be constant-time ($O(1)$), preventing additional memory pressure.

2) CPU Utilization: Execution runs on a single ARM Cortex-M4 core with no GPU or co-processor offload. The baseline task set is assumed to be schedulable in isolation with total utilization below 100%.

Because all tasks share one core, scheduling overhead must be tightly bounded. Slack computation and queue reordering are implemented with low computational cost (targeting near-$O(n)$ with respect to ready-queue size) so the wrapper does not introduce avoidable deadline risk.

V. Methodology

The project methodology is divided into an offline optimization pipeline (for benchmarking) and an online algorithm implementation. All project code and data will be version-controlled and hosted in the AtomicRadars GitHub organization repository for transparency and evaluation.

A. Optimization Pipeline (Optimization Track)

The optimization pipeline proceeds in three stages:

1) **Workload generation:** Define the task-set parameters (C_i, T_i, D_i, M_i) from STM32 profiling measurements.
2) **Offline ACO solving:** Run a Python-based Ant Colony Optimization solver using the formulation in Section III-D. Candidate schedules are constructed iteratively, and pheromone updates favor sequences with lower cumulative penalties $P(j_a, j_b)$.
3) **Baseline extraction:** Use the best offline ACO penalty as the reference *optimal* baseline for online-policy comparison.

B. Algorithm Description

The core online policy is a slack-aware wrapper around EDF. At each dispatch point, the wrapper estimates pairwise memory contention and applies deferral only when sufficient timing slack is available.

Algorithm 1: Slack-Aware Scheduling Heuristic

Input: ready queue Q, current task T_{curr}, system time t
Output: next task T_{next}
Sort Q by earliest absolute deadline (EDF order)
$T_{cand} \leftarrow Q[0]$
$Penalty \leftarrow \alpha \cdot M_{curr} \cdot M_{cand}$
if $Penalty > Threshold$ **then**
  $Slack \leftarrow d_{cand} - t - C_{cand}$
  if $Slack > SafetyMargin$ **then**
    Find $T_{alt} \in Q$ such that M_{alt} is low and T_{alt} remains schedulable
    if T_{alt} exists **then**
      return T_{alt}
    end if
  end if
end if
return T_{cand}

VI. Experimental Setup

A. Platform and Tools

The experimental environment combines an embedded target platform with offline optimization tooling:

- Hardware: STM32L452 Nucleo-64 evaluation board.
- Operating system and toolchain: FreeRTOS and STM32CubeIDE for build, deployment, and debugging.
- Profiling support: ARM Data Watchpoint and Trigger (DWT) cycle counter for microsecond-level timing measurements.
- Offline optimizer: Python with NumPy for the ACO implementation.

B. Datasets / Workloads

Evaluation uses the synthetic mixed-criticality three-task workload defined in Section III-A. To emulate memory pressure, the cryptographic task executes matrix-style memory operations and AES-like transformations on a large contiguous SRAM buffer, while the control task runs computation-dominant floating-point updates with low memory intensity.

C. Evaluation Metrics

Performance is assessed using the following metrics:

- Deadline miss ratio (DMR): percentage of released jobs that violate D_i.
- Task jitter: response-time variance of the critical control task (τ_1).
- Total contention penalty: aggregate memory-interference delay over one hyperperiod.
- Context-switch overhead: execution cost of Algorithm 1, measured in DWT clock cycles.

VII. System-Level Considerations

- Real-time constraints and latency: The proposed wrapper targets latency by reducing jitter caused by memory interference. To preserve hard real-time safety, the policy falls back to baseline EDF whenever available slack is insufficient for safe deferral.
- Energy efficiency: Although not the primary optimization objective, lowering cache and bus thrashing can reduce energy usage by decreasing high-cost memory transactions.
- Scalability: The current heuristic has approximately $O(n)$ complexity in the ready-queue size. For larger task sets, future work includes constant-time ($O(1)$) bitmap-based admission checks and lightweight hardware-assisted scheduling primitives.
- Reliability and fault tolerance: By aligning online behavior with offline-optimal ordering trends, the approach improves timing consistency under heavy memory pressure and reduces the risk of unstable runtime behavior in safety-critical deployments.

References

[1] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.

[2] H. Yun, G. Yao, R. Pellizzoni, M. Caccamo, and L. Sha, "Memguard: Memory bandwidth reservation system for efficient performance isolation in multi-core platforms," in *Proceedings of the IEEE 19th Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2013, pp. 55–64.

[3] H. Yun, R. Mancuso, Z.-P. Wu, and R. Pellizzoni, "Palloc: Dram bank-aware memory allocator for performance isolation on multicore platforms," in *Proceedings of the IEEE 20th Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2014, pp. 155–166.

[4] R. Pellizzoni, E. Betti, S. Bak, G. Yao, J. Criswell, M. Caccamo, and R. Kegley, "A predictable execution model for cots-based embedded systems," in *Proceedings of the IEEE 17th Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2011, pp. 269–279.

[5] J. M. Rivas, J. Goossens, X. Poczekajlo, and A. Paolillo, "Implementation of memory centric scheduling for cots multi-core real-time systems," in *31st Euromicro Conference on Real-Time Systems (ECRTS 2019)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 133, 2019, pp. 7:1–7:23.

[6] G. Schwäricke, T. Kloda, G. Gracioli, M. Bertogna, and M. Caccamo, "Fixed-priority memory-centric scheduler for cots-based multiprocessors," in *32nd Euromicro Conference on Real-Time Systems (ECRTS 2020)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 165, 2020, pp. 1:1–1:24.

[7] W. Lunniss, S. Altmeyer, C. Maiza, and R. I. Davis, "Integrating cache related pre-emption delay analysis into edf scheduling," in *Proceedings of the IEEE 19th Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2013, pp. 75–84.

[8] C. Lin and S. A. Brandt, "Improving soft real-time performance through better slack reclaiming," in *Proceedings of the 26th IEEE International Real-Time Systems Symposium (RTSS)*, 2005, pp. 410–421.

[9] G. Phavorin, P. Richard, J. Goossens, C. Maiza, L. George, and T. Chapeaux, "Online and offline scheduling with cache-related preemption delays," *Real-Time Systems*, vol. 54, no. 3, pp. 662–699, 2018.